Profiling report - key findings

perf - time, instructions and IPC

The optimised build runs 4.9x faster. It executes 3.0x fewer instructions, at IPC 3.56 vs 2.19. Fewer instructions is the main gain; MAQAO (below) tells whether they are vector instructions.

perf - Top-Down bottleneck classification

Optimised: 62% of slots do useful work; the main loss is Front-End Bound (20%). Front-End Bound means instruction fetch/decode cannot keep up - typically many small calls or branchy code, not memory.

perf - cache and branch misses

Every value is far below 1 miss per 1000 instructions: at this problem size the code is not limited by caches, RAM or branch prediction - the data fits in L1. Data-layout work (SoA) should therefore be measured at larger N, where it can matter.

perf - hottest functions

Optimised build: the top functions are BroadCollision::isColliding(Sphere, Sphere) (16%), PhysicsWorld::integrateVerlet(double) (15%) and Sphere::getRadius() (12%). Trivial accessors such as Sphere::getRadius() appear as separate functions: they are defined in .cpp files, so they cannot be inlined into other files without link-time optimisation (-flto).

MAQAO - hottest loops and vectorisation potential (optimised build)

0 of the 6 hottest loops of the optimised build are vectorised. CQA estimates up to 4.0x per loop if fully vectorised. Its standard workarounds for such loops include switching from arrays of structures (AoS, the current Vector3D-per-object layout) to structures of arrays (SoA).

MALT - where and how often the code allocates

3,600,068 heap allocations for 1,800,000 integration steps = 2.0 per step (10 objects). The dominant size is 264 B (100% of all allocations). Lprof attributes the operator new calls to PhysicsWorld::integrateVerlet(double). A steady-state time step should allocate nothing: these temporaries are a direct optimisation target (reuse buffers across steps). Blocks still allocated at exit: 8 (typically static objects).

Profiling report - key findings (continued)

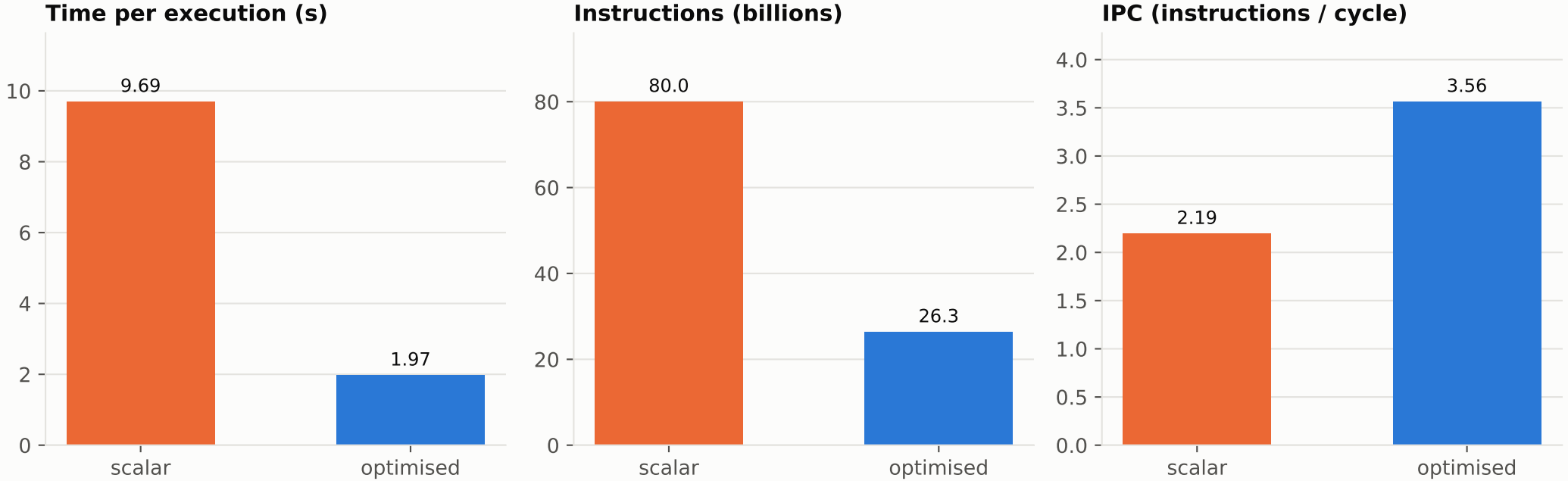
MALT - heap memory over time

Peak heap in use: 84 KiB. The curve stays flat during the run: allocations are freed as fast as they are made (no growth) - the cost is the allocation churn, not the footprint.

MAQAO - sanity check of the build profiles

scalar: 0 of 50 analysed loops vectorised, optimised: 12 of 81 analysed loops vectorised. The scalar build must show 0; otherwise its flags are not applied.

perf - time, instructions and IPC



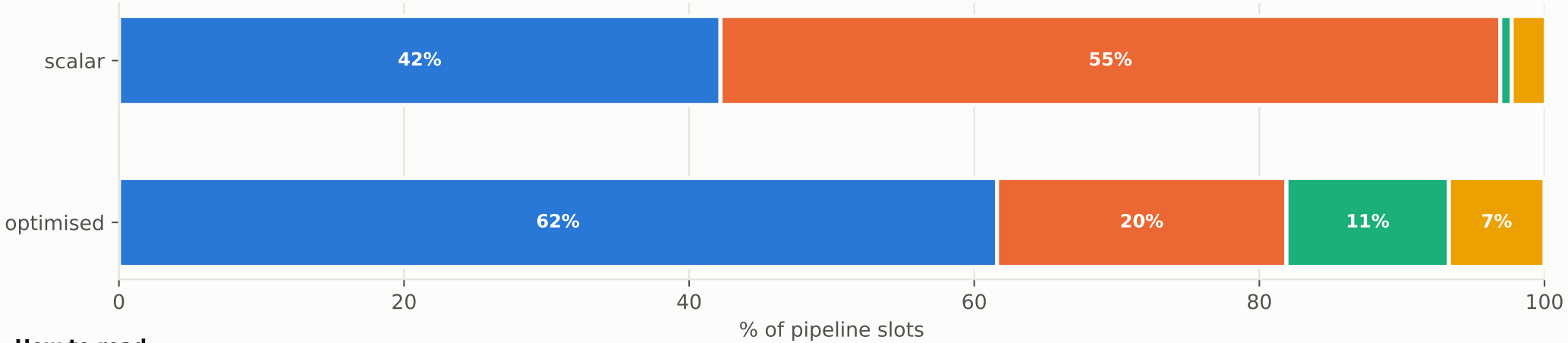
How to read

Time is the mean of 5 repeated executions (perf stat -r 5). IPC above ~3 means the pipeline is well fed; it says nothing about how much work each instruction does.

What it shows

The optimised build runs 4.9x faster. It executes 3.0x fewer instructions, at IPC 3.56 vs 2.19. Fewer instructions is the main gain; MAQAO (below) tells whether they are vector instructions.

perf - Top-Down bottleneck classification



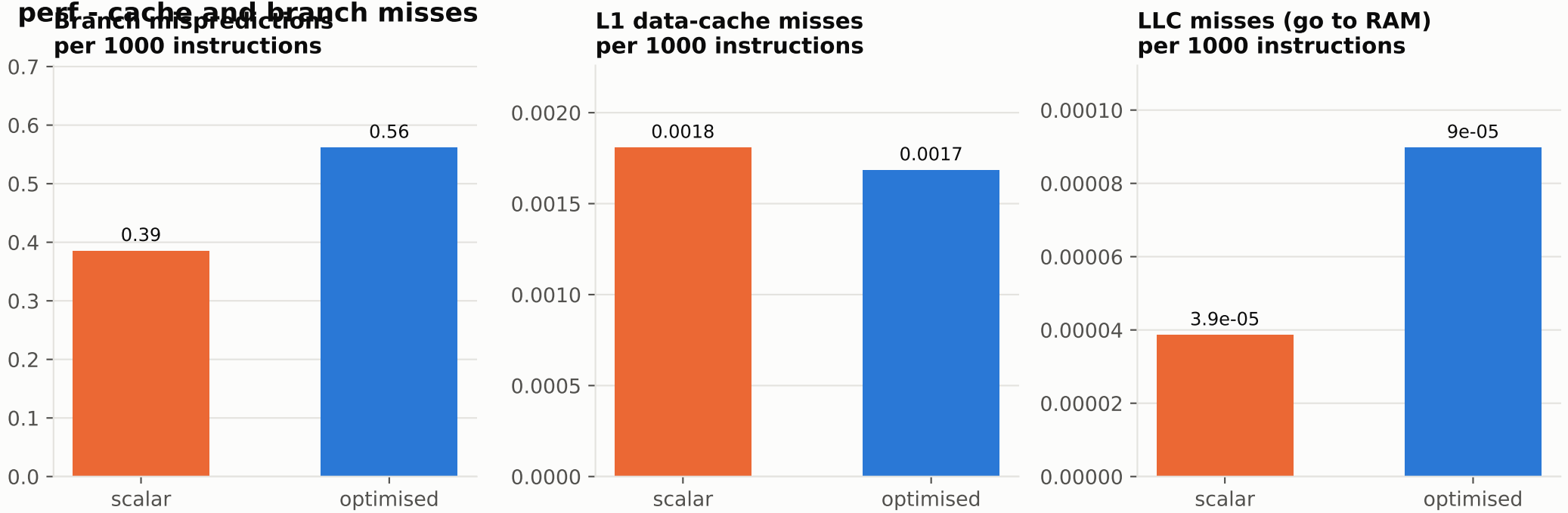
How to read

Top-Down sorts every pipeline slot into four buckets. Retiring is useful work; the other three are losses and say where to look: Front-End (code), Back-End (memory/compute), Bad Speculation (branches).

What it shows

Optimised: 62% of slots do useful work; the main loss is Front-End Bound (20%). Front-End Bound means instruction fetch/decode cannot keep up - typically many small calls or branchy code, not memory.

perf - cache and branch misses



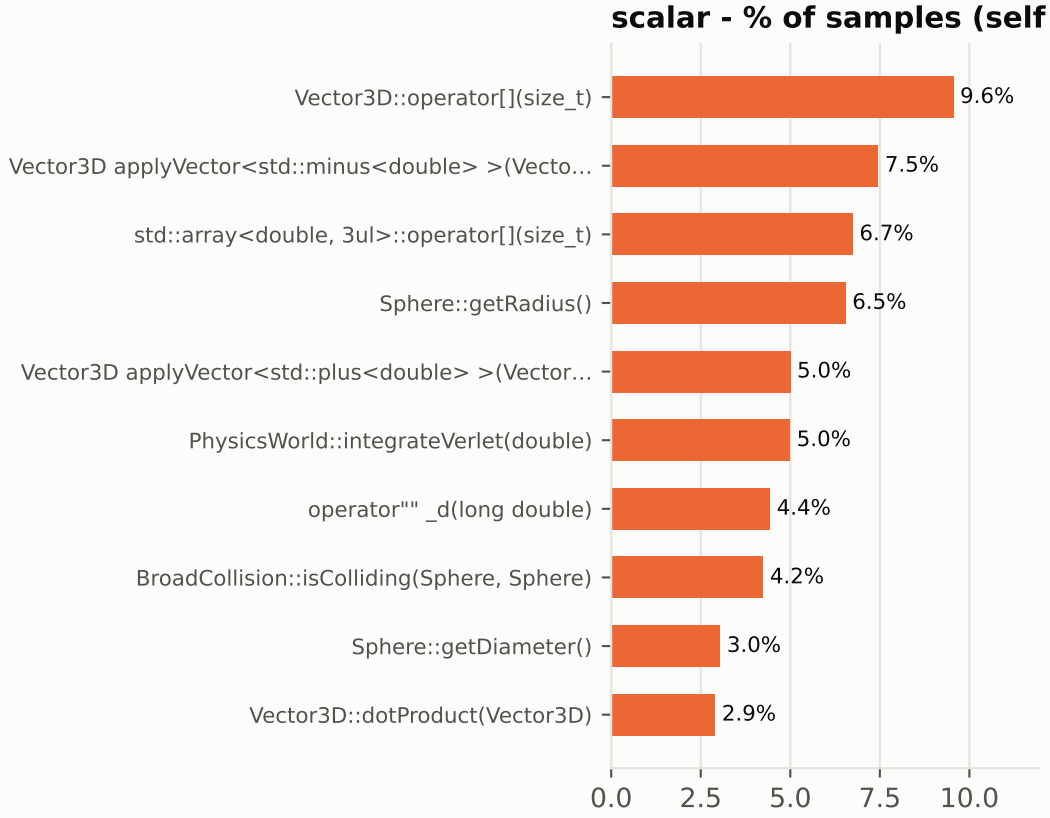
How to read

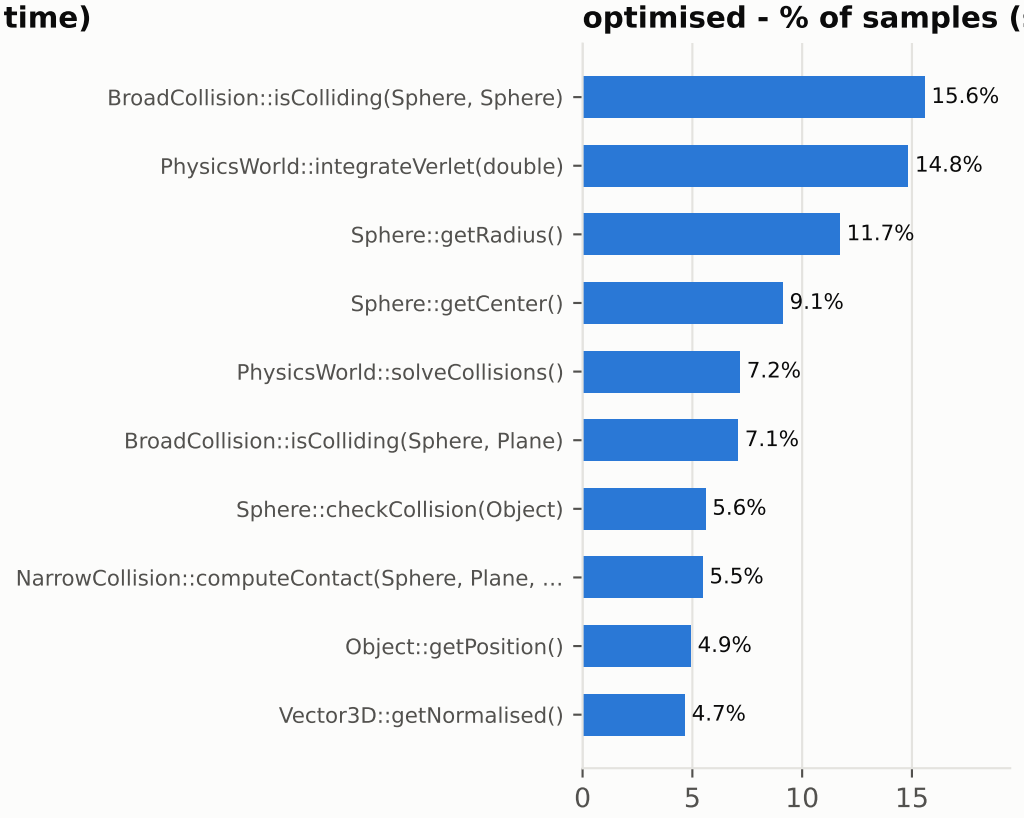
Misses per 1000 instructions (MPKI), from exact grouped counters. MPKI is preferred over a miss *rate*: a high rate on a handful of accesses costs nothing. Rough guide: LLC MPKI > 1 or branch MPKI > 5 deserve attention.

What it shows

Every value is far below 1 miss per 1000 instructions: at this problem size the code is not limited by caches, RAM or branch prediction - the data fits in L1. Data-layout work (SoA) should therefore be measured at larger N, where it can matter.

perf - hottest functions





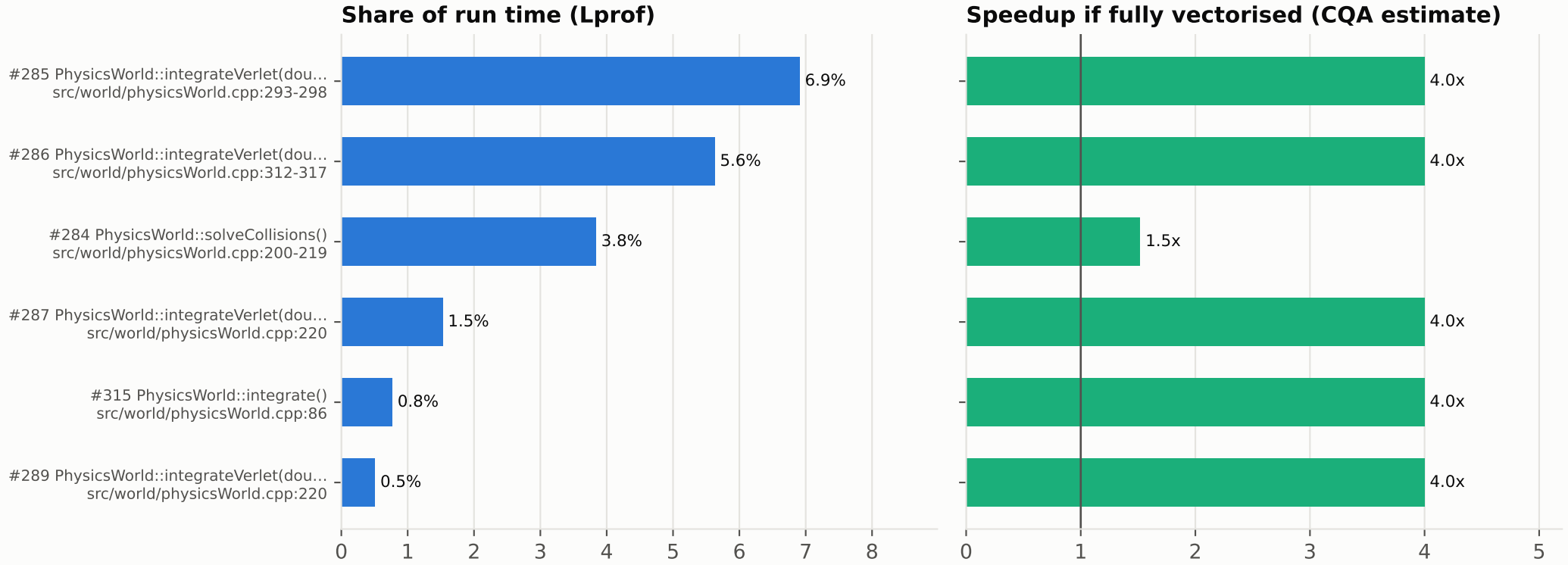
How to read

perf record samples the call stack ~1000 times per second; a function's share of samples is its share of run time (self time, callees excluded). Statistical: small shares are noisy.

What it shows

Optimised build: the top functions are BroadCollision::isColliding(Sphere, Sphere) (16%), PhysicsWorld::integrateVerlet(double) (15%) and Sphere::getRadius() (12%). Trivial accessors such as Sphere::getRadius() appear as separate functions: they are defined in .cpp files, so they cannot be inlined into other files without link-time optimisation (-flto).

MAQAO - hottest loops and vectorisation potential (optimised build)



How to read

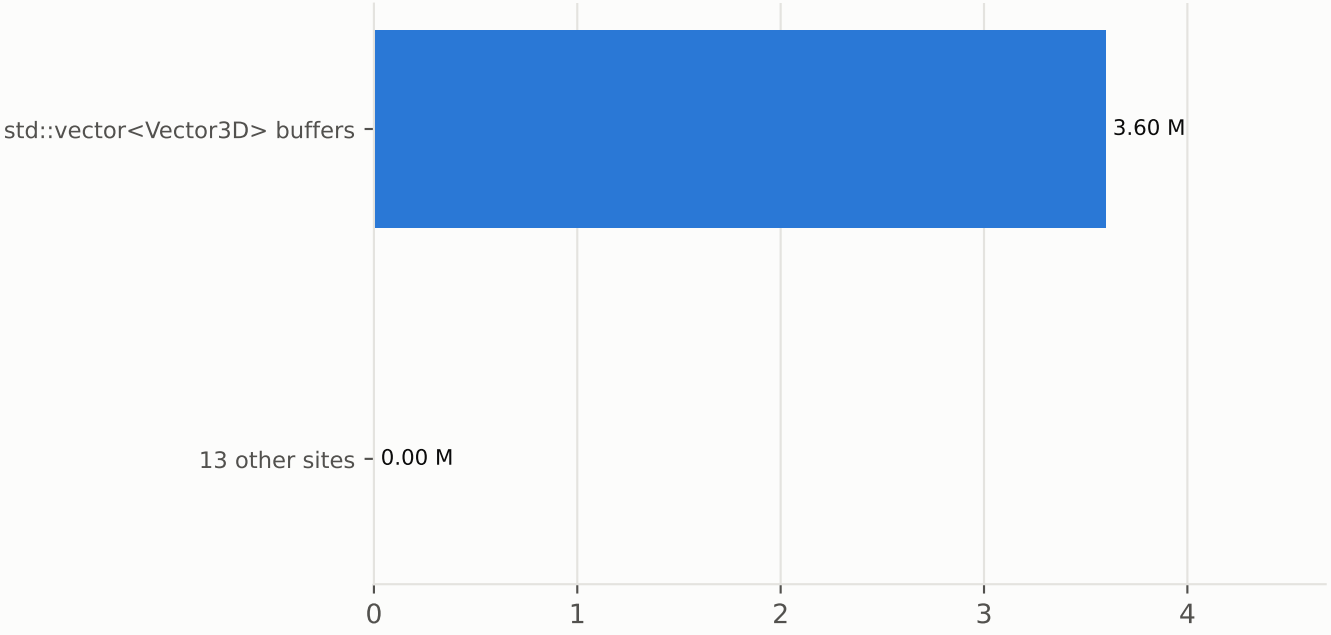
Lprof (left) measures where time goes, per source loop. CQA (right) reads the assembly of the same loop and estimates the gain if it used the full vector width. Prioritise loops that are both hot and have a large potential speedup.

What it shows

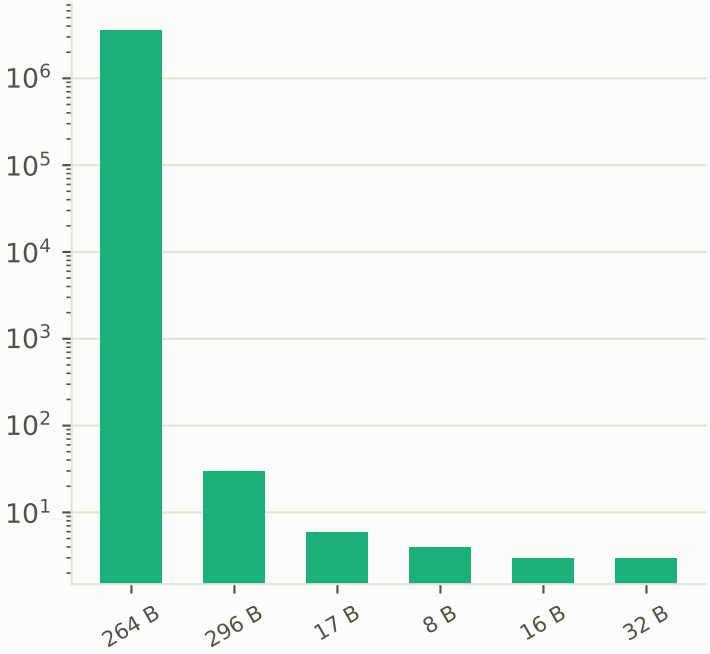
0 of the 6 hottest loops of the optimised build are vectorised. CQA estimates up to 4.0x per loop if fully vectorised. Its standard workarounds for such loops include switching from arrays of structures (AoS, the current Vector3D-per-object layout) to structures of arrays (SoA).

MALT - where and how often the code allocates

Allocations per call site (millions)



Allocations per size (log scale)



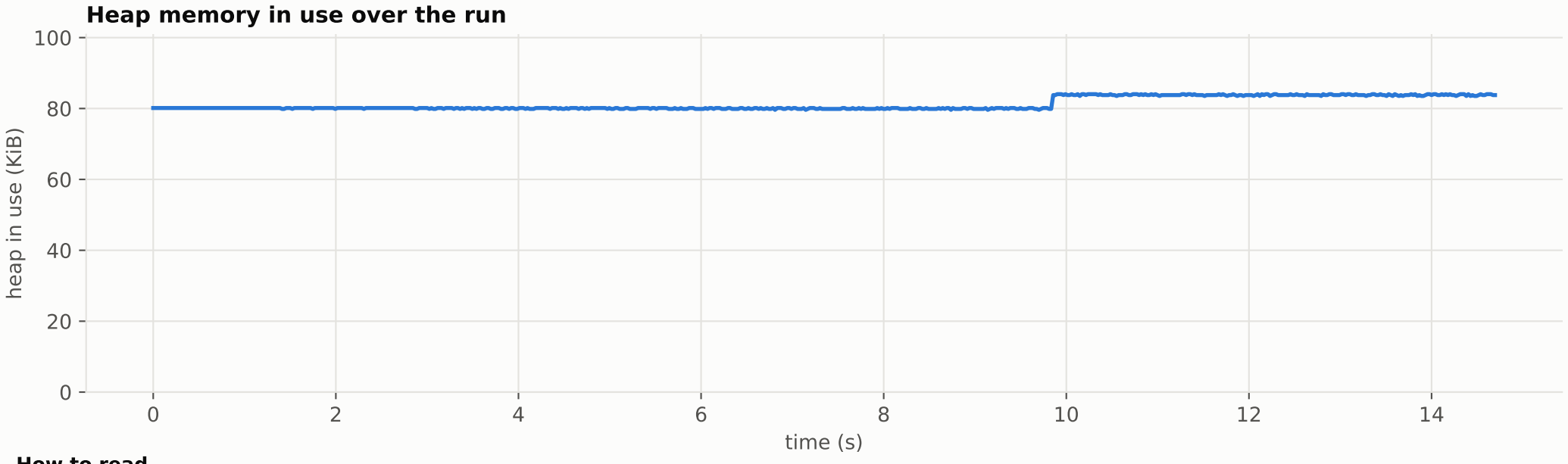
How to read

MALT intercepts every malloc/new, so counts are exact. Repeated allocations of one size inside the time loop are the pattern to remove (reuse buffers, or a pool allocator).

What it shows

3,600,068 heap allocations for 1,800,000 integration steps = 2.0 per step (10 objects). The dominant size is 264 B (100% of all allocations). Lprof attributes the operator new calls to PhysicsWorld::integrateVerlet(double). A steady-state time step should allocate nothing: these temporaries are a direct optimisation target (reuse buffers across steps). Blocks still allocated at exit: 8 (typically static objects).

MALT - heap memory over time



How to read

Bytes requested from the heap and still allocated, sampled over the run.

What it shows

Peak heap in use: 84 KiB. The curve stays flat during the run: allocations are freed as fast as they are made (no growth) - the cost is the allocation churn, not the footprint.